Évaluation pratique — Git & GitHub

Projet collaboratif de scripts d'administration système

Travail en autonomie • Groupes de 3 prioritaires (2 par exception) • **Rendu le 10 novembre 2026 avant 19h00** • avaast@myges.fr

1. Contexte professionnel

Vous êtes une équipe d'administrateurs systèmes et réseaux chargée de créer une boîte à outils de scripts pour faciliter la surveillance, l'audit et la maintenance d'un parc informatique. L'objectif principal est de démontrer votre maîtrise de Git, GitHub et du travail collaboratif, davantage que de produire des scripts complexes.

2. Constitution des équipes

Constituez vous-mêmes vos équipes. Les groupes de 3 sont prioritaires, éventuellement entre écoles. Le travail commence une fois les groupes formés. Les groupes de 2 ne sont autorisés que si l'effectif ne permet pas uniquement des trinômes. Chaque étudiant manipule personnellement Issues, Project, branches, Pull Requests et revues. En binôme, la fonctionnalité C est facultative mais toutes les compétences Git/GitHub restent obligatoires. La participation individuelle sera vérifiée.

3. Cahier des charges fonctionnel

Fonctionnalité A — Surveillance système (étudiant 1) : script affichant des informations sur l'utilisation du processeur, de la mémoire et des disques.

Fonctionnalité B — Audit local (étudiant 2) : script listant les comptes utilisateurs locaux et des informations utiles à un contrôle administratif.

Fonctionnalité C — Vérification des services (étudiant 3) : script contrôlant l'état de services définis et produisant un compte rendu. Facultatif en binôme.

Utilisez Bash et/ou PowerShell. Les scripts doivent être exécutables sur l'environnement annoncé, sans nécessiter de droits excessifs ni modifier dangereusement le système. Documentez les prérequis et commandes d'exécution. La sophistication des scripts n'est pas le critère principal.

4. Organisation du développement

Créez un dépôt GitHub commun et appliquez Git Flow, en distinguant versions stables et développements en cours. Utilisez les familles de branches étudiées et protégez les branches principales des modifications directes. Toute intégration passe par Pull Request et revue : deux approbations en trinôme, une en binôme. Si GitHub limite la protection technique, appliquez la règle manuellement et documentez cette limitation.

5. Planification avec GitHub Projects

Avant de développer, créez des Issues détaillées avec responsables, critères de réalisation, vue Kanban et Roadmap présentant l'ordre et les dépendances. Assurez la traçabilité Issues–commits–Pull Requests. Chaque étudiant participe à la gestion du Project.

6. Développement collaboratif

Développez avec des commits réguliers, explicites et liés aux tâches. Intégrez chaque fonctionnalité par Pull Request après revue. Chaque étudiant ouvre des Pull Requests et effectue des revues réelles. Au moins une Pull Request par étudiant comporte un commentaire de revue pertinent, au-delà d'une simple approbation.

7. Évolution parallèle et conflit Git

Deux étudiants travaillent en parallèle sur une partie commune du projet, par exemple un script de lancement ou un fichier de configuration partagé : **A** améliore la présentation des résultats ; **B** ajoute une option de journalisation. Ils doivent provoquer un véritable conflit Git, puis le résoudre en conservant les deux évolutions. Expliquez l'origine et la résolution dans le README. En trinôme, le troisième étudiant coordonne ; en binôme, responsabilité partagée.

8. Maintenance corrective

Après intégration, le script de surveillance affiche une information disque incorrecte ou mal formatée. Corrigez cette anomalie dans la version en cours de développement, selon le workflow étudié, avec traçabilité. Le responsable doit être différent de celui de l'incident de production.

9. Livraison de la version 1.0

Préparez une version stable avec les vérifications nécessaires, sans nouvelle fonctionnalité. Publiez le tag **v1.0**. Répercutez les éventuelles corrections de préparation dans les branches concernées et rendez la livraison vérifiable.

10. Incident après mise en production

Après v1.0, un script de vérification des services échoue lorsqu'un service contrôlé est arrêté (au lieu de signaler correctement son état). Corrigez la version de production sans intégrer les développements non validés. Publiez **v1.0.1** et répercutez le correctif dans les développements en cours. En binôme, utilisez un incident équivalent sur l'un des scripts obligatoires.

11. Exigences de participation individuelle

Chaque étudiant crée et manipule des branches, synchronise le dépôt, réalise plusieurs commits/push, gère des Issues et le Project, ouvre des Pull Requests, participe aux revues et aux intégrations. Répartissez équitablement conflit, maintenance et livraison. Les notes individuelles peuvent différer.

12. Documentation attendue

Le README présente le projet, les scripts et leurs prérequis, les commandes d'exécution, les responsabilités, le workflow, les étapes de développement, le conflit et sa résolution, les versions et les difficultés.

IDENTIFICATION OBLIGATOIRE : indiquez explicitement la correspondance entre le nom, le prénom et le pseudonyme GitHub de chaque membre :

Nom	Prénom	Pseudonyme GitHub	Rôle
Nom 1	Prénom 1	@pseudo1	Étudiant 1
Nom 2	Prénom 2	@pseudo2	Étudiant 2
Nom 3	Prénom 3	@pseudo3	Étudiant 3

ATTENTION : l'absence de correspondance pseudonyme GitHub / nom / prénom dans le README entraîne automatiquement la note de 0/20 pour l'ensemble du groupe.

13. Questions de synthèse

Répondez brièvement dans le README, avec une répartition équitable et l'identification de l'auteur de chaque réponse.

1. Quel est l'intérêt de séparer développements en cours et versions stables ?
2. Pourquoi imposer une revue de code avant intégration ?
3. Quelles situations provoquent un conflit Git et pourquoi sa résolution n'est-elle pas toujours automatique ?
4. Quelle différence entre correction classique et correction urgente de production ?
5. Pourquoi répercuter une correction de production dans les développements en cours ?
6. Quel est le rôle d'une branche de release ?
7. Comment GitHub Projects et les Issues facilitent-ils organisation et traçabilité ?
8. Comment retrouver l'origine d'une modification dans l'historique GitHub ?

14. État final et livrables

Scripts fonctionnels et documentés ; Git Flow et protections ; familles de branches, Pull Requests et revues vérifiables ; conflit réel résolu ; correction en développement, release, hotfix et tags v1.0/v1.0.1 ; synchronisation des corrections, Issues, Project et README conformes. Les branches temporaires fusionnées peuvent être supprimées si leur historique reste consultable.

15. Modalités de rendu

Date limite impérative : aujourd'hui, mardi 10 novembre 2026, avant 19h00. Un seul mail par groupe à **avaast@myges.fr**, objet « Évaluation Git/GitHub — Groupe X ». Indiquez les noms et prénoms des membres, le lien du dépôt GitHub et le lien du GitHub Project. Les deux ressources doivent être publiques. Aucun ZIP accepté.

16. Barème

Critère	Points
Organisation et répartition du travail	2
GitHub Projects, Issues et Roadmap	4
Git Flow, branches et historique Git	4
Pull Requests, protections et Code Reviews	3
Gestion d'un conflit réel	2
Corrections, release et hotfix	3
Documentation et traçabilité	2
TOTAL	**20**

La notation tient compte du résultat, de la méthode, de la traçabilité et de la participation individuelle.